Software Requirements Specification

Smart Farmer Assistance Platform

A Crop-Lifecycle-Centered Mobile and Web Platform for Personalized Farm Management and Assistance

Version 0.1 – Week 1 Draft

Prepared for: Web Programming and Mobile Application Development

Department of CSE, Khulna University, Khulna

Document drafted with AI-assisted requirement writing; reviewed and edited by the project team.

Prepared by

Team members:

Jeyasmen Akter

Student ID: 220221

Sohag Chandra

Student ID: 220238

Prepared for

Dr. Kazi Masudul Alam

Professor

Computer Science and Engineering Discipline

Khulna University, Khulna

Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Smart Farmer Assistance Platform, a crop-lifecycle-centered mobile application with a supporting admin web system for Bangladesh. It is intended to guide design, implementation, and evaluation of the system across the course project.

1.2 Scope

Smart Farmer Assistance Platform is not a generic agricultural information app. Its core purpose is to manage an individual farmer's crop journey end-to-end: capturing farm and crop details, generating a stage-based cultivation plan, scheduling and reminding farming activities, delivering stage-appropriate and weather-aware guidance, recording diary entries and expenses, assisting with crop problem reports, and preserving a complete crop history through harvest.

A companion admin web panel allows an agricultural administrator to maintain the underlying knowledge that powers the farmer-facing experience — supported crops, crop stages, activity templates, weather-alert rules, and the crop-problem knowledge base — without requiring code changes.

The system is developed following an AI-in-the-loop engineering approach, where AI agents assist across the software development lifecycle while humans remain responsible for final decisions at each checkpoint (see Section 6).

1.3 Objectives

1. Reduce the gap between what a crop needs at a given moment and what the farmer actually knows to do, by generating a stage-based cultivation plan automatically at crop registration.
2. Improve the quality of day-to-day farming decisions by delivering stage-appropriate, weather-aware guidance rather than relying solely on informal sources such as neighbors or local dealers.
3. Ensure timely action on farming activities through a reminder and automatic stage-advancement engine, so activities are not missed or performed too late.
4. Keep agricultural knowledge maintainable and data-driven, through an admin-managed crop / stage / activity-template / knowledge-base model, so new crops or updated guidance do not require code changes.
5. Preserve a complete, queryable crop history and expense record for every crop cycle, so a farmer can review what was done, what it cost, and what happened, even after harvest.
6. Provide an accessible first-response assistance channel for crop problems, using text description and/or an uploaded photo, with a clear disclaimer that results are possibilities requiring expert confirmation, not confirmed diagnoses.
7. Keep the system usable on low-end Android devices and under intermittent connectivity, syncing core records once back online.
8. Demonstrate an AI-in-the-loop software engineering workflow, where AI agents assist at each SDLC phase (requirements, architecture, coding, testing, documentation) subject to a mandatory human checkpoint.

1.4 References

- Plantix (PEAT GmbH) — AI-assisted crop pest/disease diagnosis mobile app
- e-Krishi Batayon / Bangladesh Department of Agricultural Extension (DAE) advisory services
- Kisan Suvidha (India) — farmer advisory and weather-alert mobile app
- FarmLogs / Climate FieldView — crop-cycle and farm-record management platforms
- OpenWeatherMap API documentation — weather data source referenced for this project

2. Overall Description

2.1 Product Perspective

Smart Farmer Assistance Platform is a new, standalone system with a three-part architecture: a Farmer Mobile App (primary interface for farmers), an Admin Web Panel (for managing agricultural master data), and a Backend API/Database that serves both clients and integrates with an external Weather API. The system separates master data (Crop, CropStage, Activity templates — maintained by Admin) from instance data (FarmerCrop, ActivityLog, DiaryEntry, Expense, Problem — generated as a farmer uses the app), so agricultural knowledge can be updated without disturbing a farmer's already-generated plan history.

2.2 Product Functions (Summary)

- Farmer account registration and secure login
- Farm record management (location, land size, soil type, irrigation availability)
- Crop registration with automatic, stage-based cultivation plan generation
- Automatic crop-stage advancement based on elapsed days since planting
- Activity scheduling with Today's / Upcoming views and reminder notifications
- Stage-appropriate, weather-aware farming guidance
- Weather forecast display and weather-triggered farming alerts
- Farming diary for recording performed activities, cost, and notes
- Cultivation expense tracking and crop-wise financial summary
- Crop problem reporting via text/image with knowledge-base-driven assistance
- Complete crop history preserved through and after harvest
- Farmer dashboard summarizing crops, activities, weather, expenses, and problem status
- Admin web panel for managing crops, stages, activity templates, and the problem knowledge base

2.3 User Classes and Characteristics (User roles)

- Farmer — primary mobile app user; manages own farms, crops, activities, diary, and expenses, and submits crop-problem reports. Assumed to have basic smartphone literacy and intermittent mobile connectivity.
- Admin — manages crop, stage, and activity-template master data and the problem knowledge base via the web panel. Assumed to have domain/agricultural familiarity and access to a desktop/laptop browser.

2.4 Operating Environment

- Mobile App: Flutter or React Native (single codebase for Android/iOS), primary farmer-facing client
- Admin Web: React.js with a component library (e.g., MUI or Ant Design) for management screens
- Server: Node.js + Express (REST API), deployable on standard cloud/free-tier hosting
- Database: PostgreSQL (relational — fits the structured crop/stage/activity model), with MongoDB as an alternative if document-based modeling is preferred
- External Weather API: OpenWeatherMap or an equivalent free-tier weather provider
- Notifications: Firebase Cloud Messaging (FCM) for push reminders and alerts

2.5 Design and Implementation Constraints

The team size is two members over an approximately 2.5-month (10–11 week) timeline; the technology stack is chosen for that constraint and, once selected, is kept fixed during implementation to avoid losing time to re-tooling.

- Crop, stage, and activity-template data must remain admin-configurable so that adding a new supported crop does not require a code change.
- Weather-dependent features (guidance, alerts) shall degrade gracefully — core activity tracking, diary, and expense features shall continue to function if the Weather API is temporarily unreachable, using cached weather data where available.
- Crop-problem assistance is knowledge-base-driven for the core build; any machine-learning-based image disease detection is out of scope for the core deliverable and is listed as an optional feature (Section 3.7, and Appendix B).
- Core mobile features shall function correctly under intermittent connectivity, syncing to the backend once connectivity is restored.

2.6 Assumptions and Dependencies

- A farmer registers and manages crops through the mobile app; the admin web panel is not farmer-facing and requires separate admin credentials.
- Weather data is location-derived from the farm's registered location and is cached and periodically refreshed to reduce external API calls and cost.
- Activity schedules and stage-advancement timing are derived entirely from admin-defined durations and templates; the system assumes this master data is populated before farmers register crops that depend on it.
- The crop-problem knowledge base is assumed to be manually curated by the admin for the crops supported at launch; automated/ML-based diagnosis is a future upgrade path, not a current dependency.

3. System Features (Functional Requirements)

3.1 Authentication & Farmer Profile Management

ID	Requirement	Priority
REQ-UM-01	System shall allow a farmer to register using name, phone number/email, and password.	High
REQ-UM-02	System shall allow a registered farmer to log in and log out securely.	High
REQ-UM-03	System shall allow a farmer to view and update profile information (name, contact, location, photo, language preference).	Medium
REQ-UM-04	System shall allow an admin to log in through a separately authenticated admin panel; admin accounts are provisioned by the project team, not self-registered.	High
REQ-UM-05	System shall authenticate all sessions using JWT tokens and store passwords using industry-standard hashing (e.g., bcrypt).	High
REQ-UM-06	System shall keep a farmer's data private and accessible only to that farmer and authorized admins.	High

3.2 Farm Management

ID	Requirement	Priority
REQ-FM-01	System shall allow a farmer to add one or more farms.	High
REQ-FM-02	Each farm record shall capture location, land size, soil type, and irrigation availability.	High
REQ-FM-03	System shall allow a farmer to edit or remove a farm.	Medium
REQ-FM-04	System shall list all farms belonging to a farmer with a summary of active crops per farm.	Medium

3.3 Crop Registration & Lifecycle Tracking

ID	Requirement	Priority
REQ-CP-01	System shall allow a farmer to select a crop from a supported crop list for a given farm.	High
REQ-CP-02	System shall require the planting/start date and relevant cultivation details at crop registration.	High
REQ-CP-03	System shall automatically generate a crop-specific cultivation plan (stages and activities) based on the selected crop and its admin-defined stage/activity templates.	High
REQ-CP-04	System shall allow a farmer to maintain multiple active crops across different farms simultaneously.	Medium

ID	Requirement	Priority
REQ-CP-05	System shall track the current lifecycle stage of each registered crop (e.g., Land Preparation, Planting, Seedling, Vegetative, Flowering, Fruiting, Harvest).	High
REQ-CP-06	System shall automatically advance a crop's stage based on elapsed days since planting, using admin-defined stage durations for that crop.	High
REQ-CP-07	System shall display the current stage and day-count (crop age) to the farmer.	Medium

3.4 Activity Scheduling, Reminders & Guidance

ID	Requirement	Priority
REQ-AS-01	System shall generate a schedule of farming activities (irrigation, fertilizer, weeding, pest/disease monitoring, etc.) for each crop stage, derived from admin-managed activity templates.	High
REQ-AS-02	System shall allow the schedule to be re-calculated if the farmer updates crop information.	Medium
REQ-AS-03	System shall display a list of activities due today for each active crop.	High
REQ-AS-04	System shall allow a farmer to mark an activity as Pending, Completed, or Skipped.	High
REQ-AS-05	System shall display upcoming activities for the next 7 days with the relevant due date.	Medium
REQ-AS-06	System shall send a reminder notification when a scheduled activity is due or approaching.	High
REQ-AS-07	System shall generate stage-appropriate guidance/recommendations for a farmer's active crop, considering crop type, crop age, current stage, and available weather data.	High
REQ-AS-08	Notifications and guidance text shall be crop-specific, actionable, and shall avoid generic, non-actionable messages.	Medium

3.5 Weather Integration & Alerts

ID	Requirement	Priority
REQ-WA-01	System shall fetch weather information (temperature, rainfall probability, humidity, forecast) for a farm's registered location via an external Weather API.	High
REQ-WA-02	System shall generate farming alerts derived from weather conditions (e.g., drainage check before heavy rain).	Medium
REQ-WA-03	System shall cache weather data and refresh it periodically to reduce external API calls and continue serving cached data if the Weather API is temporarily unreachable.	Medium

3.6 Farming Diary, Expenses & Crop History

ID	Requirement	Priority
REQ-DH-01	System shall allow a farmer to record farming activities performed (irrigation, fertilizer, pesticide, weeding, observations) with date, quantity, cost, and notes.	High
REQ-DH-02	Diary entries shall be linked to the specific crop and farm; a farmer shall be able to view, edit, or delete an entry.	Medium
REQ-DH-03	System shall allow a farmer to record cultivation expenses by category (seed, fertilizer, pesticide, irrigation, labour, transport, other).	High
REQ-DH-04	System shall automatically calculate total cultivation cost per crop and display a crop-wise financial summary across all of a farmer's crops.	High
REQ-DH-05	System shall maintain a complete digital history for each crop — start date, current/previous stages, completed activities, problems, and expenses — viewable at any time, including after harvest.	High
REQ-DH-06	System shall allow a farmer to mark a crop as Harvested and record harvest date and quantity, preserving full crop history afterward.	Medium
REQ-DH-07	System may calculate revenue and profit if a selling price is provided at harvest.	Optional

3.7 Crop Problem Reporting & Assistance

ID	Requirement	Priority
REQ-PA-01	System shall allow a farmer to report a crop problem via text description and/or an uploaded image.	High
REQ-PA-02	System shall match the report against the admin-curated problem knowledge base and return a possible issue, likely symptoms, possible causes, and recommended actions.	High
REQ-PA-03	System shall clearly present results as possibilities, not confirmed diagnoses, and shall recommend expert confirmation where appropriate.	High
REQ-PA-04	System shall store reported problems in the crop's problem history with a status (e.g., Monitoring, Resolved).	Medium
REQ-PA-05	Image-based automated disease detection (machine-learning model) is an optional enhancement and is out of scope for the core knowledge-base-driven build.	Optional

3.8 Admin Panel & Knowledge Base Management

ID	Requirement	Priority
REQ-AD-01	System shall provide a web-based admin panel with separate authentication from the farmer app.	High
REQ-AD-02	Admin shall manage supported crops, crop stages, and activity templates.	High

ID	Requirement	Priority
REQ-AD-03	Admin shall manage the crop-problem knowledge base (issue name, symptoms, causes, recommended actions).	High
REQ-AD-04	Admin shall manage weather-alert rules and registered farmer/user accounts.	Medium
REQ-AD-05	Admin actions (add/edit/deactivate a crop, stage, or activity) shall reflect in farmer-facing plans for new or updated crops without requiring a code change.	High

3.9 Farmer Dashboard

ID	Requirement	Priority
REQ-DB-01	System shall present a dashboard summarizing active crops, today's activities, upcoming activities, current weather, latest expense total, and recent problem status.	High
REQ-DB-02	The dashboard shall answer, at a glance, "what is happening with my crop" and "what do I need to do next."	Medium

4. External Interface Requirements

4.1 User Interfaces

- Farmer mobile app: onboarding, farm/crop registration forms, crop lifecycle view, Today's/Upcoming activity lists, guidance screen, weather & alerts, diary, crop history, problem reporting, expense tracker, dashboard
- Admin web panel: crop/stage/activity-template management, problem knowledge-base management, farmer/user management, weather-alert rule management

4.2 Hardware Interfaces

- Device camera (photo capture for crop-problem reports and profile photo)
- Device GPS / location services (farm location capture for weather lookup)
- Device push-notification channel (activity reminders and weather alerts)

4.3 Software Interfaces

- Weather API (e.g., OpenWeatherMap) for forecast data and weather-triggered alerts
- Firebase Cloud Messaging (FCM) for push notifications
- Cloud image storage (e.g., Firebase Storage / Cloudinary free tier) for crop-problem images
- Relational database (PostgreSQL) for master and instance data

4.4 Communication Interfaces

- HTTPS/REST between the mobile app / admin web and the backend API
- HTTPS/REST from the backend to the external Weather API

- Push messaging (FCM) from backend to mobile client for reminders and alerts

5. Non-Functional Requirements

Category	Requirement
Performance	Dashboard and activity screens shall load within 2–3 seconds under normal mobile network conditions.
Security	Passwords shall be stored using industry-standard hashing (e.g., bcrypt); all API traffic shall use HTTPS/TLS; JWT session tokens shall expire and be refreshed securely.
Data Privacy	A farmer's farm, crop, diary, and expense data shall be accessible only to that farmer and authorized admins.
Usability	The mobile app shall use a simple, farmer-friendly interface with support for Bangla and English, and shall remain usable on low-end Android devices.
Reliability	Core features (activity tracking, diary, expenses) shall function correctly even with intermittent connectivity, syncing when back online; weather-dependent features shall degrade gracefully using cached data if the Weather API is unreachable.
Scalability	The backend architecture shall support adding new crops, stages, and activity templates without code changes (data-driven configuration through the admin panel).
Maintainability	Code shall be modular (by feature/module) with clear separation between API, business logic, and data layers.
Portability	The mobile application shall run on both Android and iOS through a cross-platform framework (Flutter or React Native).
Availability	The backend API and admin panel shall target reasonable uptime suitable for an academic demonstration deployment (e.g., cloud/free-tier hosting).

6. System Architecture Overview

The system follows a 3-tier client-server architecture layered over a data-driven master/instance data model: a presentation tier (mobile app + admin web), an application tier (backend REST API containing the plan-generation, stage-advancement, guidance, and reminder logic), and a data tier (relational database plus the external Weather API integration).

6.1 Client Layer

- Farmer Mobile App — Flutter or React Native (primary, current build)
- Admin Web Panel — React.js, for agricultural master-data management

6.2 Application Layer

- Authentication API → Farm/Crop API → Activity Scheduling Engine → Guidance Engine → Notification Service
- Plan generation and stage-advancement logic runs as a deterministic, data-driven process against admin-managed templates, not hard-coded per crop

- The system depends on one primary external API — the Weather API — with cached-data fallback if unreachable

6.3 Data Layer

- Relational database: PostgreSQL, separating master data (Crop, CropStage, Activity templates) from instance data (FarmerCrop, ActivityLog, DiaryEntry, Expense, Problem)
- Cloud object storage for crop-problem images and profile photos

6.4 External Integration Layer

- Weather API (e.g., OpenWeatherMap) resolves a farm's registered location to current and forecast weather data, feeding both the guidance engine and the alert generator
- Weather responses are cached per location key and refreshed periodically, so core activity, diary, and expense features remain unaffected by short external-API outages
- Firebase Cloud Messaging delivers reminder and alert notifications generated by the application layer to the farmer's device

7. Appendix

7.1 Appendix A: Glossary

Term	Definition
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement
UC	Use Case
REQ-XX-##	Requirement identifier format used throughout this SRS (module prefix + sequence number), e.g. REQ-CP-01
Crop Stage	A defined phase in a crop's growth cycle, e.g., Planting, Flowering, Harvest
Activity Template	An admin-defined farming task tied to a specific crop stage (e.g., irrigation, fertilizer)
FarmerCrop	A specific instance of a crop planted by a farmer on a farm (as opposed to the crop master record)
Master data	Admin-maintained reference data (Crop, CropStage, Activity template, Knowledge Base) that defines what the platform knows
Instance data	Farmer-generated data (FarmerCrop, ActivityLog, DiaryEntry, Expense, Problem) created as the platform is used
Knowledge Base	The admin-curated set of crop-problem entries (issue, symptoms, causes, recommended actions) used to assist problem reports
JWT	JSON Web Token — used for session authentication
FCM	Firebase Cloud Messaging — push-notification service used for reminders and alerts
AI-in-the-Loop Engineering	A development approach where AI agents assist across the SDLC, with mandatory human review at each checkpoint

7.2 Appendix B: Analysis Models

At the time of this document's current version, the following analysis models are planned but not yet finalized:

- Entity-Relationship Diagram (ERD) — covering Farmer, Farm, Crop, CropStage, FarmerCrop, Activity, ActivityLog, DiaryEntry, Expense, Problem, ProblemKnowledgeBase, Notification, and Admin. Scheduled for Week 2 alongside the API contract.
- System architecture diagram — visual representation of the client, application, data, and external-integration layers described in Section 6. To be finalized and embedded in this section in the next SRS revision.
- Data flow diagram (DFD) — tracing a crop's path from registration through plan generation, stage advancement, and activity completion to crop history; planned for Week 2.

These will be added to this appendix as they are completed; this SRS version documents their absence as a known gap rather than omitting mention of them.

7.3 Appendix C: Issues List

#	Issue	Status
1	Final weather API provider (OpenWeatherMap vs. an alternative free-tier provider) has not been confirmed for production accuracy in Bangladesh-specific locations.	Open — pending verification
2	Whether the mobile app targets Flutter or React Native has not been finalized between the two team members.	Open — pending team decision
3	Image-based disease detection (ML model) is documented as optional; whether time permits even a prototype version by Week 10 is uncertain.	Open — monitored
4	Crop-problem knowledge base seed data (symptoms, causes, recommended actions for the initially supported crops) has not yet been collected.	Open
5	Activity template durations and stage-duration data for the initially supported crops have not yet been collected/validated against local agronomic sources.	Open
6	Whether offline-first sync (for intermittent connectivity, REQ-DH/AS features) is fully implemented or best-effort for the course timeline is unconfirmed.	Open — design decision pending
7	License for the project (open-source vs. restricted) has not been decided.	Open
8	System architecture diagram and data flow diagram (Appendix B) are not yet embedded in this document.	Open — scheduled Week 2